NDN Log Analysis Using Big Data Techniques: NFD Performance Assessment

Conference Paper · March 2018

DOI: 10.1109/BigDataService.2018.00032

3 authors:

Junior Dongo
Université Paris-Est Créteil Val de Marne - Université Paris 12
7 PUBLICATIONS **8** CITATIONS
SEE PROFILE

Charif Mahmoudi
National Institute of Standards and Technology
30 PUBLICATIONS **78** CITATIONS
SEE PROFILE

Fabrice Mourlin
Université Paris-Est Créteil Val de Marne - Université Paris 12
41 PUBLICATIONS **78** CITATIONS
SEE PROFILE

NDN Log Analysis using Big Data Techniques: NFD Performance Assessment

Junior DONGO*, Charif MAHMOUDI*, Fabrice MOURLIN*
*University of Paris Est, Creteil, France
Algorithmic, Complexity and Logic Laboratory, Creteil-Paris, France
Email: junior.dongo@u-pec.fr, charif.mahmoudi@lacl.fr, fabrice.mourlin@u-pec.fr

***Abstract*—Named Data Networking (NDN) is presented as the future Internet Architecture. It brings several properties, including security and content dissemination. The measurement of such properties needs a deep analysis of forwarding behavior during the runtime. The analysis presented in this paper stands on two pillars: the logs/traces generation and the big data analysis. This paper proposes an instrumentation framework composed by an extension of the dumping tool *ndndump*. This extension generates big data friendly logs that are essential to capture the behavior of NDN forwarding and tools for hardware performance measurement. Based on these logs, the second part of this work is a discussion on some properties of NDN using big data techniques that show the performance and content dissemination. We explore a potential solution where the NFD (Named Data Networking Forwarding Daemon) code might be reused to build a high-performance forwarder. The main goal is the NFD performance evaluation. Execution scenarios are realized using the emulator *minindn* and compared to native network scenarios.**

***Keywords*-NDN; Big Data; Log analysis; Measurement; NFD**

I. INTRODUCTION

Applications such as web servers, file servers, while running, generate messages regarding the application evolution such as the status information, warning, or errors [1]. These messages are stored in files and are used to predict events, to help fixing problems when they occur, and to improve system engineering tasks [2]. In the network domain, logs are often produced by collecting network traffic. This is generally done by dumping exchanged information between nodes for analysis purposes. To be able to retrieve useful information from these logs, some analysis must be performed to exploit them. Traditional log analysis [3] can be challenging when dealing with network log analysis. In fact, logs size can increase quickly, especially when performing network dumps. On the other hand, we can have many small files which in general represent a large amount of data. Dumps performed at each node can be merged on a single node for the analysis part. But, moving data that have large size across the network consumes bandwidth and decreases the network performances. With logs distributed on the nodes across the network, we must find a way to analyze them.

Part of the Information Centric Networking (ICN), NDN (Named Data Networking) brings a new way of communication [4]. Moving from a host centric networking to a content centric networking, it brings many features, such as network scalability and in-network caching [5]. In this paper, we analyze interest and data evolution over the time based on big data techniques and evaluate NDN forwarder performance. This is done by extending the dumping tool of NDN and providing a performance measurement tool for hardware. Interest and data are the two types of packets exchanged over the network. Consumers send Interest requests for data content, which are satisfied by Producers. NDN also allows data retrieval from close hosts when available, instead of forwarding the interest to the Producer. This is done by caching data related to an interest at a node, using in-network caching.

NFD (Named Data Networking Forwarding Daemon) is a network forwarder implementing the NDN protocol. It forwards (decides whether, when, and where to forward) an Interest based on a forwarding strategy [6]. This is one of the main component of the NDN protocol.

Our contributions are the assessment of NFD performance in emulated and native environmement. We then proposed a load balancing algorithm for NFD scalability.

The remainder of this paper is organized as following: In Section II, we highlighted related works, followed by the methodology applied in Section III. Details about our experimentation concerning NDN network log collection is presented in Section IV. In Section V, we presented and discussed our results. Potential solution for the NFD limitation is presented in Section VI. Finally, Section VII, concludes and presents future research possibilities.

II. RELATED WORK

Log analysis has been employed for a long time and a lot of algorithms have been proposed to facilitate the analysis. Marcin et al. in their multidimensional log analysis proposed an algorithm based on text mining algorithms which checks words and phrases frequencies to identify interesting events [3]. They unified the logs into a unique format, used regular expression to fill data into a database. Based on this approach, they have created a knowledge

database to detect well-known event pattern, such as cyber-attacks. This approach is suitable for application event log analysis but it is hard to apply when dealing with network dumps.

Simulation and emulation have been for long time good ways to run networks experiments. In fact, they allow the execution of multiple scenarios for testing and validation purposes quickly, without the need to buy costly network equipments. In network wireless domain, R. Karri et al. have showed through simulation that the energy consumed by data transactions varies linearly with data size and non-linearly with data rate [7]. In military communications [8], L. Zhang et al. studied the Armys Warfighter Information Network-Tactical (WIN-T) and the Navy's Automated Digital Networking System (ADNS) through emulations. They showed that NDN provides better efficiency in terms of average delivery delay, maximum delivery delay and bandwidth consumption than TCP-Based FTP.

Network log events are file containing what happened over the network during a period. It helps understand and solve problems when they occur. M. Melody et al. using log analysis based on both machine learning and pattern matching method could detect SQL injection on a web server [9].

In our work, we combine the advantages of both emulation and log analysis. We used emulation to produce log files on which we performed big data analysis.

III. METHODOLOGY

In this part, we present the different tools used in this work, our network topology on which the experimentation has been conducted and our approach.

A. Tools

The experimentation tools were chosen to allow researchers working on NDN to reproduce our work easily.

NDN provides a set of tools. The dumping tool *ndndump* [10] was created to provide a tcpdump-like tool for Named Data Networking (NDN), by dumping data traffic between nodes over a NDN network. Unfortunately, the output format of this tool is not adapted to big data analysis framework. Our first task was to provide an extension of this tool to make the output compatible with big data analysis frameworks. This extension [11] is compatible with both native and emulated networks. This tool generates a comma separated csv file with headers. In general, all the packets have the same output. However, some fields are not applicable for Data or Interest packets. Note that the fields that do not apply to a type of packet will be empty.

The Interest Packets fields description:

`Timestamp,Source,Destination,TunnelType, PacketSize,PacketType,Name,n/a,MustBeFresh, Nonce,InterestLifetime`

The Data packet fields description:

`Timestamp,Source,Destination,TunnelType, PacketSize, PacketType,Name, FreshnessPeriod,n/a,n/a,n/a`

To be able to generate traffic over the network, we used a more realistic scenario. To store the data the consumer request, we use a repository application at the producer nodes. The Repository new generation (*Repo-ng*) is an implementation of NDN persistent in-network storage [12] which allows the user to store named data over NDN network. *Repo-ng* uses *sqlLite3* databases to store the data. It supports classical data objects manipulation, such as insert and delete. The retrieve (or read) operation is performed via named data transfer. In fact, once a repository node receives an interest for a data, it searches for the data in the local storage using the interest name. Data are sent back as a NDN data packet if the corresponding data are available.

For our experiment, we have constructed a NDN network using *mini-ndn* [13]. *mini-ndn* is a network emulator based on mini-net which is for IP based network emulation. *mini-ndn* extends *mini-net* and include features specific to NDN. Instead of *ndnSim* [14] which is a simulation tool for NDN, we have chosen *mini-ndn* because it offers the possibility to create realistic virtual networks, run real application code. It is also challenging to run ndndump on ndnSim. Finally, experimentation under *mini-ndn* are easily reproducible. We used Hadoop for the big data analysis. Hortonworks and Cloudera provide virtual machines with Hadoop components installed, ready to use for experiments and for production. We use a Cloudera VM because of our familiarity with the tool and its ease of use. We also used Rhadoop [15] which is a collection of R packages to run R scripts on HDFS using the MapReduce paradigm.

B. Network topology

We wanted to have multiple possible paths to satisfied an interest. We have used a partial meshed network for reachability purpose.

For example, content for video1 can be retrieved by consumer1 using these possible paths:

- producer_video1 – S5 – consumer1
- producer_video1 – S2 – S4 – consumer1
- producer_video1 – S2 – S1 – consumer1

All the links between components are 100Mbps. Figure 1 shows the network schema.

We have the following settings:

- 3 producers will produce video content: /lacl/video[1-3]
- 1 producer for an audio content: /lacl/audio
- 5 consumers sending interest requests for audio and video content. This number will increase to perform a scalability analysis.

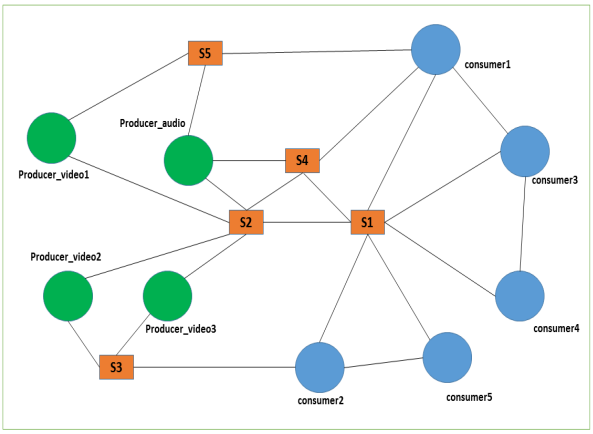

Figure 1. Network topology

C. Approach

The data were collected by setting up a network in *mini-ndn* to run the experiments. We then carried out a dumping at each node, stored the logs in files on the host file system. The data were made available to HDFS into the Cloudera VM using a shared folder between the VM and the host file system as shown in Fig.2. Finally, we used *R scripts* to analyze the data. This was made possible using *Rhadoop* in *MapReduce* paradigm for the execution. *rmr2* and *rhdfs* are *R* packages which permited the execution.

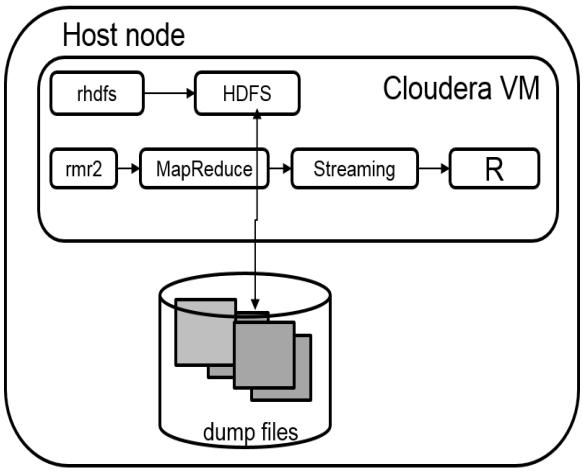

Figure 2. Cloudera VM communication HDFS - host File System

IV. EXPERIMENTATION

For an experiment based on the topology described in Section III, the host operating system must have the whole NDN stack installed, including repo-ng and our modified version of ndndump. We provided a script, available on github [16] to automate the installation process. *mini-ndn* performs using command line. It requires a network topology file and an experiment file. The first step was the creation of the network topology. *mini-ndn* provides a graphical tool to design the network topology and then convert it into a topology file. We wrote our experiment in Python. The forwarding strategy provides the intelligence to make decision on whether, when, and where to forwarding Interests [6]. NFD comes with 6 default forwarding strategies. We set the forwarding strategy to "best route strategy" for all the nodes. The best route strategy forwards an Interest to the upstream with lowest routing cost. To make the content available on the producers, we provided video and audio source contents, using repo-ng. These contents were loaded in the repository using this command for video 1 :

```
ndnputfile /lacl/video1/1 /lacl/video1
video1.avi
```

and made available for a request by advertising the name at the producer side. Consumer nodes executed a shell script for interest requests which were sent using the example of consumer1:

```
ndngetfile -o output.avi /lacl/video1
```

and were based on a Poisson distribution (the frequency and the number of requests had been generated randomly based on this distribution cf. Table I).

$$\lambda_{frequency} = 100$$
$$\lambda_{number} = 1000$$

We then started the ndndump application to capture data exchanged between nodes. All the dump files were located on the host computer which eased their management. There were two phases in our experimentation. We first execute for the previous topology, and then run a set of experiments in which we started with one consumer and incremented the number. Our simulations run for 12 hours and we got 450 GB of data to analyze. We have used the open source hypervisor Virtualbox to deploy the Cloudera VM. Virtualbox offers a way to share folders between the virtual machines and the host. We have exploited this functionality to make our data available anywhere. It was then easy to load the data into HDFS.

V. RESULTS AND DISCUSSION

This section shows results obtained through an emulation. Moreover, the validity of those results is checked by performing tests over a native NDN network with physical machines.

Nodes	Nb. Interest requests	Frequency (ms)	Interest
Consumer1	971	143	/lacl/video1
Consumer2	946	125	/lacl/audio
Consumer3	981	110	/lacl/video2
Consumer4	1006	113	/lacl/video3
Consumer5	1034	123	/lacl/video1

Table I
INTEREST REQUESTS DISTRIBUTION

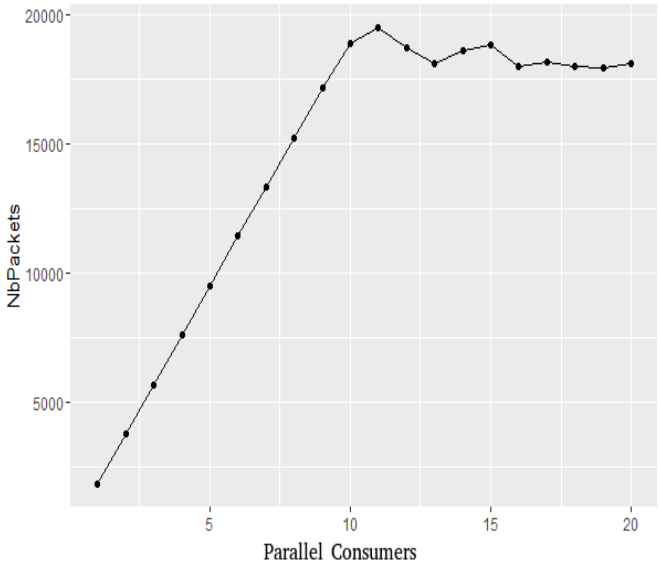

Figure 3. Mean packet per consumers

A. Emulation

Our emulation has been performed using the configuration described in the experimentation section. We have performed 20 runs in which we increased the number of consumer at each run starting from 1 consumer.

As we can see in Fig.3, the number of packets evolves linearly until 10 consumers and reaches a peak at 11 consumers. After that, the number of packet decreases a little bit to become stable. This decrease implies a loss of packets, a deterioration of performance, a drop of additional packets at the forwarder side. This could be explained by the fact that the forwarder is no longer able to forward packet because it has reached its maximal forwarding capability. For a detailed analysis and a better understanding of our previous result, we investigated the cause of this result. Fig.4 shows the number of packet forwarded per second. We ignore the first two seconds because results are not meaningful, because we are at the beginning of the experiments. We can see that there is a linear evolution for the entire duration of the experiment. From 11 consumers, the graph is close to a sinusoid. Peaks show that the forwarder has received a lot of packets, due to congestion, and subsequent packets are drooped. This explains the decease of the processing packets.

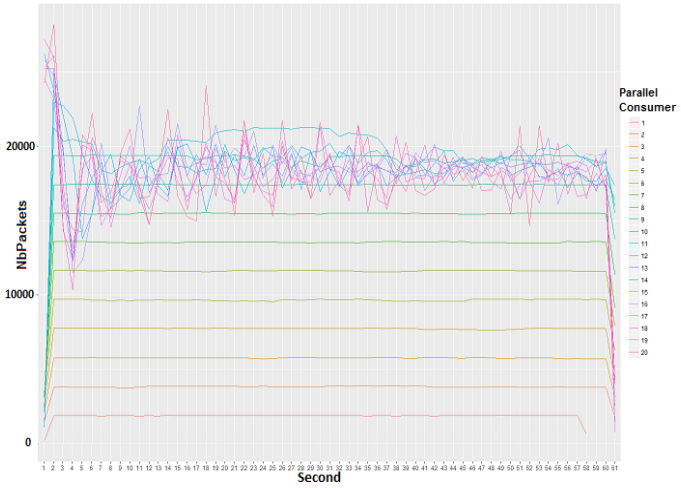

Figure 4. Packets per second (pps) plots

When the number of packets to process decreases, we have an increasing of the received number of packet until the next saturation. This phenomenon repeats during the entire experiment. The emulation shows that when reaching 20000 packets, NFD starts dropping incoming packets. Looking for the cause of this behavior, the next step was to perform the experiment with tests on a native network. We measured CPU and memory utilization and suspected these to be the main cause.

B. Test NDN native network

We have performed the same analysis with dump from a native NDN network.

In our experiment environment, the physical machines had the following settings: a Quad/core 3.4 GHz processor, 8 GB of memory, 500 GB of disk, and 10 Gigabits Ethernet connection. All the nodes run the NFD (Named Data Networking Forwarding Daemon). Fig.5 shows the machine interconnection and the tools used on each machine.

Node-1 runs ndnping client which is used to send interest requests. We have instantiated from 1 to 100 ndnping consumers. Each consumer sends one packet per millisecond with the following parameters: InterestLifeTime = 4000, MustBeFresh = true and FreshnessPeriode = 0.

Node-2 runs our modified version of ndndump. We also belt a script to monitor network IO, CPU and Memory usage. On Node-3, we installed the ndnping server tool which will produce data packets for interests sent from Node-1. We have instantiated from 1 to 100 ndnping servers on this node. For each iteration, we restarted the NFD, incremented the consumer count, started the network dumping, started the consumers, waited 60 second, stoped the consumers and the dump.

Figure 6 presents our findings. This plot represents the mean number of incoming packets from Node-1 to Node-2 (face1) and the mean number of outgoing packets from

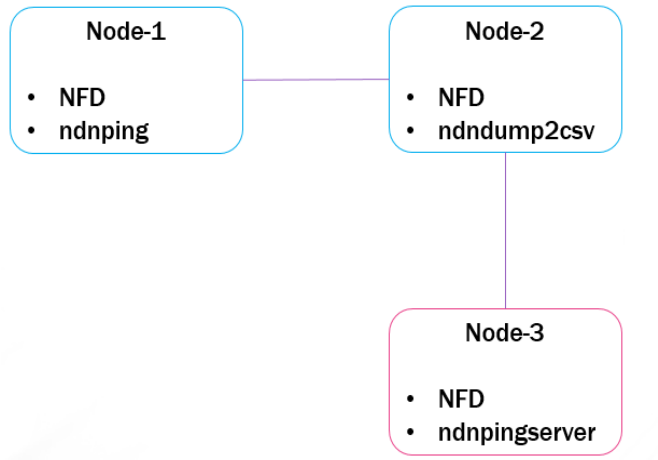

Figure 5. Configuration

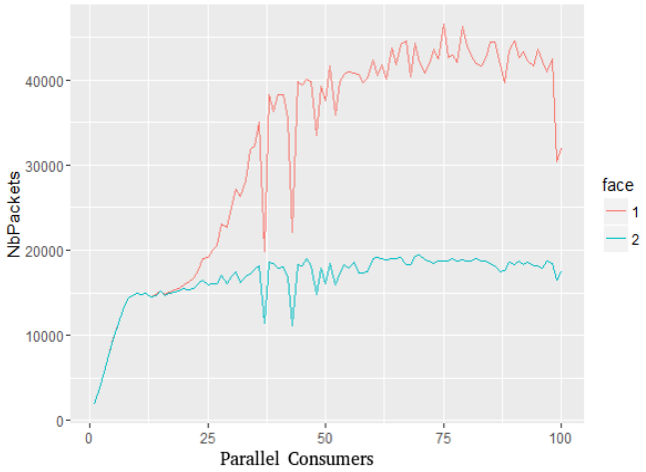

Figure 6. Mean packet from native network experiment

Node-2 to Node-3 (face 2). When the number of incoming packets goes beyond 20000 packets per second, the forwarder forwards only around 20000, which means that it drops the other packets. We can see that we obtain the same result on a native network with the one from our emulated network. Looking for the reasons of the limitation of the NFD, we intuitively thought that it was due to the CPU and the memory.

Fig.7 shows the average percentage of memory usage. As we can see, less than 5% of the memory is used. We can conclude that the limitation is not related to the memory. In Fig.8, we have the mean CPU usage, and we can see that the maximum CPU usage is 75% and is only one core. This concludes that the CPU is not the cause of the limitation.

The test on a native NDN network confirmed the results of our emulation. The main cause of this behavior is the performance of the NFD.

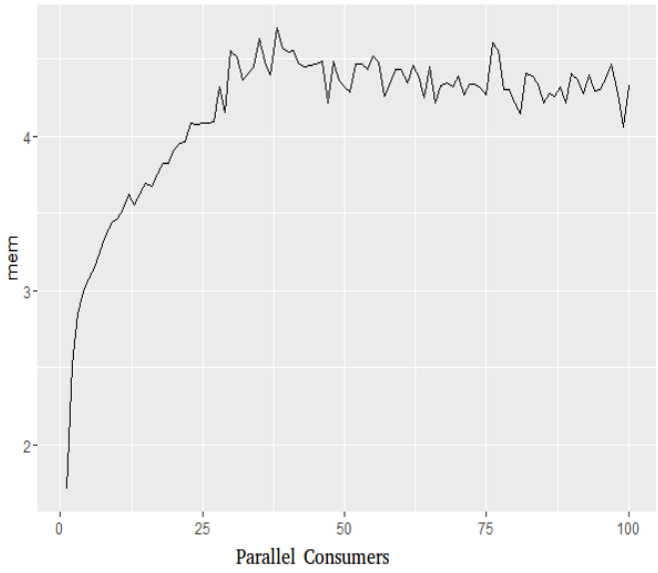

Figure 7. Mean % of memory usage (8GB)

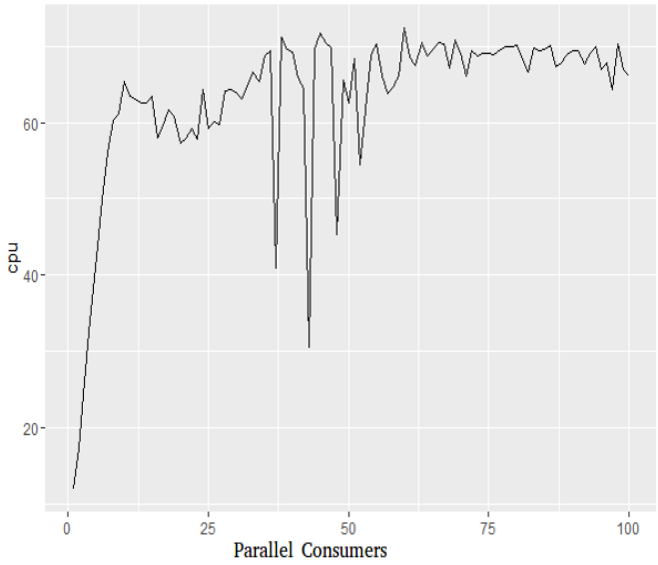

Figure 8. Mean % of CPU usage

VI. EXPLORING POTENTIAL SOLUTION FOR THE NFD LIMITATION

We performed a test as presented in Fig.9. Our approach was the virtualization of the NFD. We have installed multiple instances of the NFD on the same machine. A VFD (Virtual Forwarder Deamon) is a NFD running inside a VM.

The algorithm (Algorithm 1) starts by provisioning the FIB (Forwarding Information Base) of each vfd using a central component like the function assured by the NLSR (Named Data Link State Routing Protocol) [17]. This component will be responsible using a hash function for the

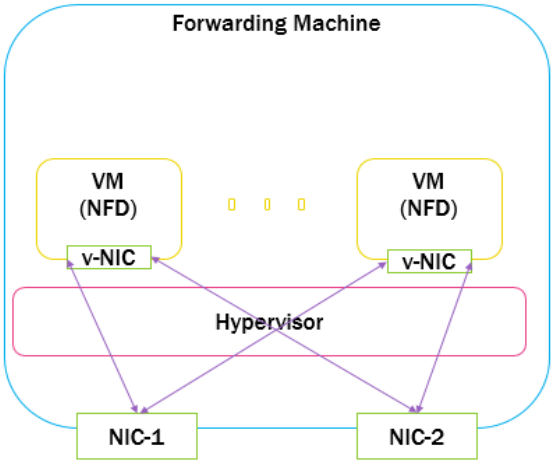

Figure 9. NFD virtualization

load balancing which will be an aggregation of function corresponding to the name in the FIB of every NFD. Depending on the number of entry in the FIB, several VFD instances will be deployed for the packet processing. For example, if we have 3 names entry in the FIB; two instances of VFD will be deployed. To avoid a name to be present in more than one VFD, a subset of name is associated to each VFD. There is also an association of hash function for the pattern matching in the load balancer. When a packet arrives at a face, the hash function is applied on the name to determine on which VFD it will be sent. The names are automatically moved between the VFDs and hash functions are updated accordingly to balance the load depending on the traffic.

This solution permits to double the number of forwarded packets but creates a loss at NDN benefits. In fact, if two interests for the same content arrive and are managed by two different VM, both will be forwarded, because of the lack of sharing PIT information among the VM. A PIT-less approach [18], if successful in combination with our attempt, could be a possible solution. The idea will be to provide an optimization of the current implementation instead of developing from scratch.

```
OnPacket(packet){
  dest_hach = hash(prefix)
  If(global_fib **excludes** dest-hach){
   target_fib = longest_prefix_match(prefix)
   global_fib <- global_fib +
         fibentry(hash(prefix), target_fib)
  }
  SendTo(packet, global_fib[dest_hach])
  UpdatePrefixsStatistics()
}
```

```
RebalanceFibs(listFibs, listStats){
  Clean(global_fib)
  CurrentLoad = sum(listStats)
  Index = 0
  For stats in listStats {
    Counter = 0;
    For fib in listFibs{
      While (Counter < CurrentLoad
          / count(list_fibs)){
        Prefix = getPrefix(stats)
        AddFibEntryOnVM(fib, prefix)
        Add(global-fib, hash(prefix),fib)
        Counter += getLoad(Stats)
      }
    }
  }
}
```

Algorithm 1. Load Balancing algorithm

VII. CONCLUSION AND FUTURE WORK

In this study, we analyzed the data evolution in time over a NDN network. We first extended the dumping mechanism in NDN. We have used our extension of ndndump on an emulated network. Using big data analysis, the results show that from a behavioral point of view, NFD has a limitation around 20.000 packets. We developed measurement tools for performance analysis in NDN and performed experiments on a native NDN network which confirmed our findings but also showed that the limitation is not due to the memory or the CPU. The main cause is related to the actual forwarder implementation. We explored a potential solution for solving the limited forwarding performance by using load balancing on virtual forwarders. Our next study will be a test of CICN (Community ICN) and then the implementation of a framework for ICN networks monitoring based on big data analysis techniques.

REFERENCES

[1] A. Oliner, A. Ganapathi, and W. Xu, "Advances and challenges in log analysis," *Communications of the ACM*, vol. 55, no. 2, pp. 55–61, 2012.

[2] J. Wang, C. Li, S. Han, S. Sarkar, and X. Zhou, "Predictive maintenance based on event-log analysis: A case study," *IBM Journal of Research and Development*, vol. 61, no. 1, pp. 11–121, 2017.

[3] M. Kubacki and J. Sosnowski, "Multidimensional log analysis," in *Dependable Computing Conference (EDCC), 2016 12th European*. IEEE, 2016, pp. 193–196.

[4] L. Zhang, D. Estrin, J. Burke, V. Jacobson, J. D. Thornton, D. K. Smetters, B. Zhang, G. Tsudik, D. Massey, C. Papadopoulos *et al.*, "Named data networking (ndn) project," *Relatório Técnico NDN-0001, Xerox Palo Alto Research Center-PARC*, 2010.

[5] N. Team, *Named Data Networking (NDN)*. [Online]. Available: https://named-data.net/project/archoverview/

[6] A. Afanasyev and al., *NFD Developers Guide*. [Online]. Available: http://named-data.net/wp-content/uploads/2016/10/ndn-0021-7-nfd-developer-guide.pdf

[7] R. Karri and P. Mishra, "Modeling energy efficient secure wireless networks using network simulation," in *Communications, 2003. ICC'03. IEEE International Conference on*, vol. 1. IEEE, 2003, pp. 61–65.

[8] B. Etefia, M. Gerla, and L. Zhang, "Supporting military communications with named data networking: An emulation analysis," in *MILITARY COMMUNICATIONS CONFERENCE, 2012-MILCOM 2012*. IEEE, 2012, pp. 1–6.

[9] M. Moh, S. Pininti, S. Doddapaneni, and T.-S. Moh, "Detecting web attacks using multi-stage log analysis," in *Advanced Computing (IACC), 2016 IEEE 6th International Conference on*. IEEE, 2016, pp. 733–738.

[10] N. Team, *ndndump*, https://github.com/named-data/ndn-tools/tree/master/tools/dump.

[11] C. Mahmoudi, *Modified version of ndndump*. [Online]. Available: https://github.com/charifmahmoudi/ndndump

[12] N. Team, *Repo protocol and repo-ng*. [Online]. Available: https://redmine.named-data.net/projects/repo-ng/wiki

[13] ——, *mini-ndn*. [Online]. Available: https://github.com/named-data/mini-ndn

[14] S. Mastorakis, A. Afanasyev, I. Moiseenko, and L. Zhang, "ndnsim 2.0: A new version of the ndn simulator for ns-3," *NDN, Technical Report NDN-0028*, 2015.

[15] *RHadoop*. [Online]. Available: https://github.com/RevolutionAnalytics/RHadoop/wiki

[16] C. Mahmoudi, *ndn-stack*. [Online]. Available: https://github.com/charifmahmoudi/ndn-stack

[17] A. Hoque, S. O. Amin, A. Alyyan, B. Zhang, L. Zhang, and L. Wang, "Nlsr: named-data link state routing protocol," in *Proceedings of the 3rd ACM SIGCOMM workshop on Information-centric networking*. ACM, 2013, pp. 15–20.

[18] C. Ghali, G. Tsudik, E. Uzun, and C. A. Wood, "Living in a pit-less world: A case against stateful forwarding in content-centric networking," *arXiv preprint arXiv:1512.07755*, 2015.